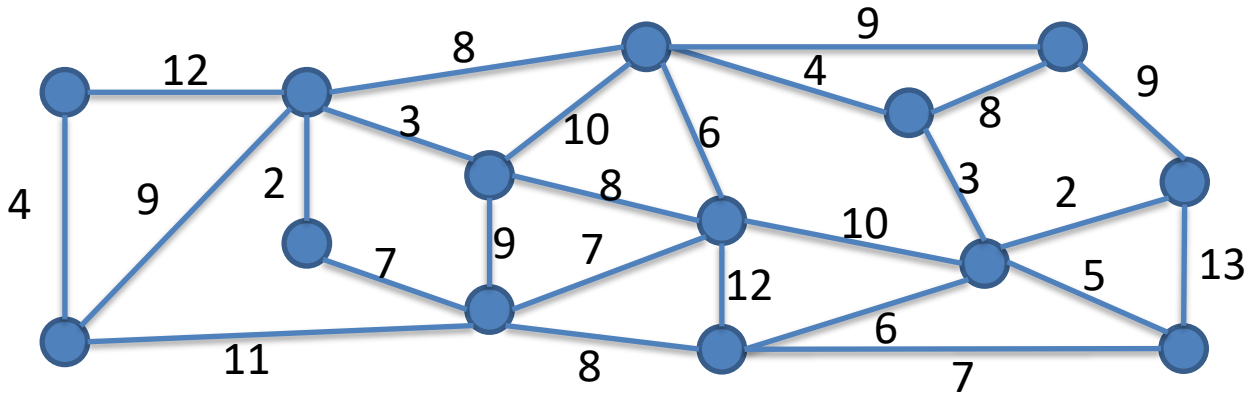
TCSS 343 – Design and Analysis of Algorithms

Minimum Spanning Tree

Graph Problems

- Recall a weighted (undirected or directed) graph is an ordered triple, $G = (V, E, w)$, comprised of a set of vertices V, a set of edges E, and a weight function w such that:
  - $V = \{v_1, v_2, \ldots, v_n\}$ is the set of $n \geq 0$ vertices.
  - $E \subseteq \{\{u, v\} | u, v \in V\}$ is the set of edges.
  - $w: E \to \mathbb{R}$ is a function that maps from the edge set to a real number.

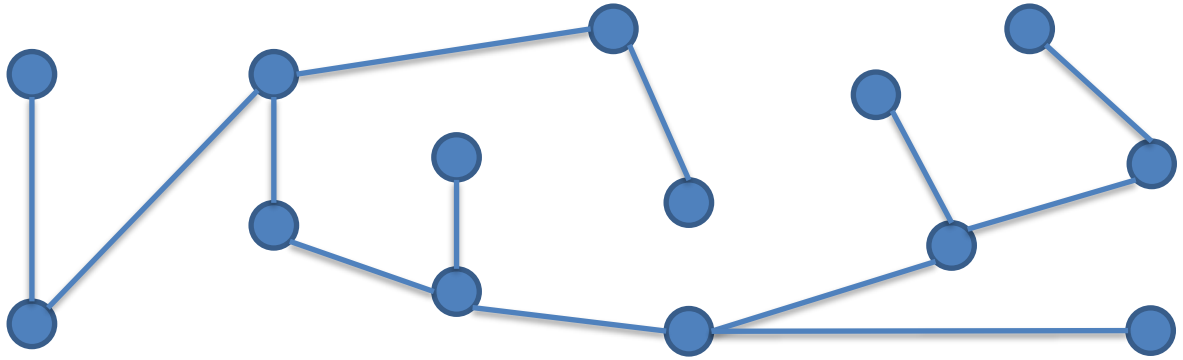


Trees

- Recall that an undirected graph is connected if there exists a path from u to v for any pair of vertices $u, v \in V$.
- Recall a tree is a graph with no cycles.

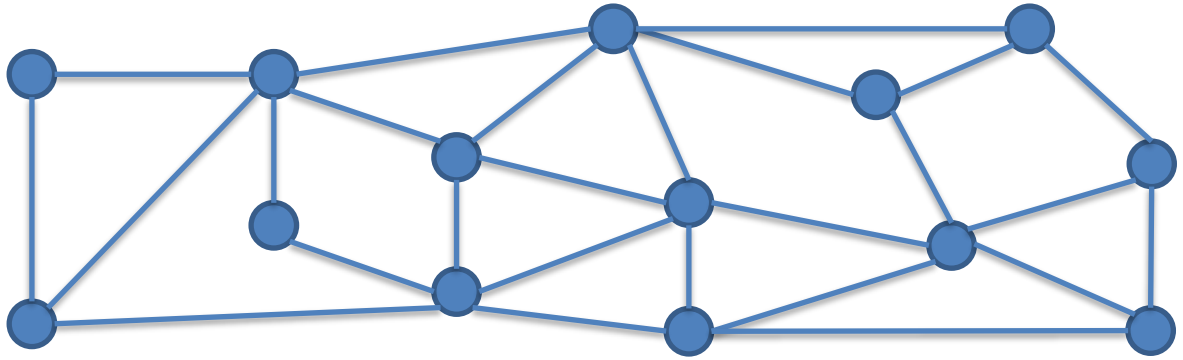
```
Tree Detection (TREE)
Input: G = (V,E) a connected undirected graph
Output: "yes" if the graph is a tree and "no" otherwise

Cycle Detection (Cycle)
Input: G = (V,E) a connected undirected graph
Output: "yes" if the graph has a cycle and "no" otherwise
```



Spanning Trees

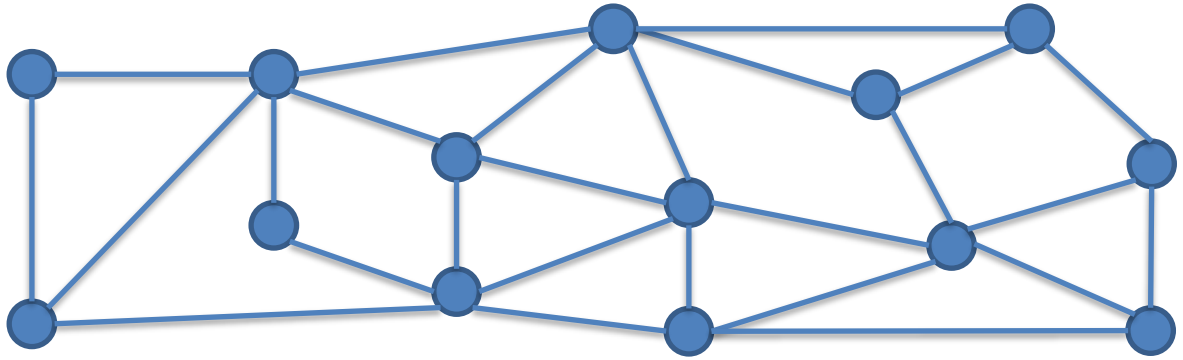
- A spanning tree of a graph is a tree that is formed using all the nodes of the graph and a subset of the connections.
  - Spanning_tree
  - There are often many spanning trees of a graph.
  - How can we create a spanning tree from a graph?
    - Delete edges from the original graph.
    - Add edges to an empty version of the graph.



Spanning Trees

- A spanning tree of a graph is a tree that is formed using all the nodes of the graph and a subset of the connections.

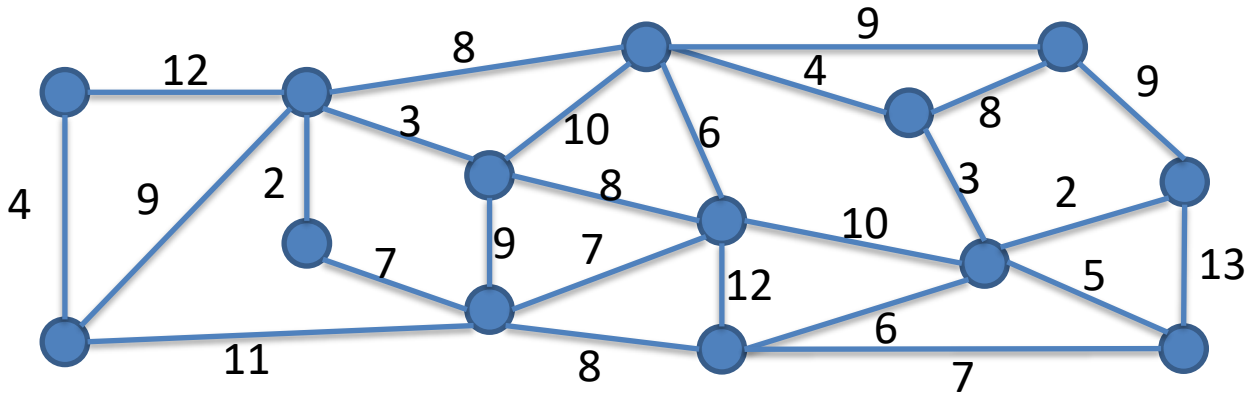
Spanning Tree (SPAN)
Input: $G = (V, E)$ a connected undirected graph
Output: $T = (V, E')$ a spanning tree of G



Minimum Spanning Trees

- When we have a weighted graph we can measure the weight of a spanning tree by adding the weights of all edges in the tree.
  - This spanning tree has weight 87.
  - Can we find a spanning tree with weight less than 87?

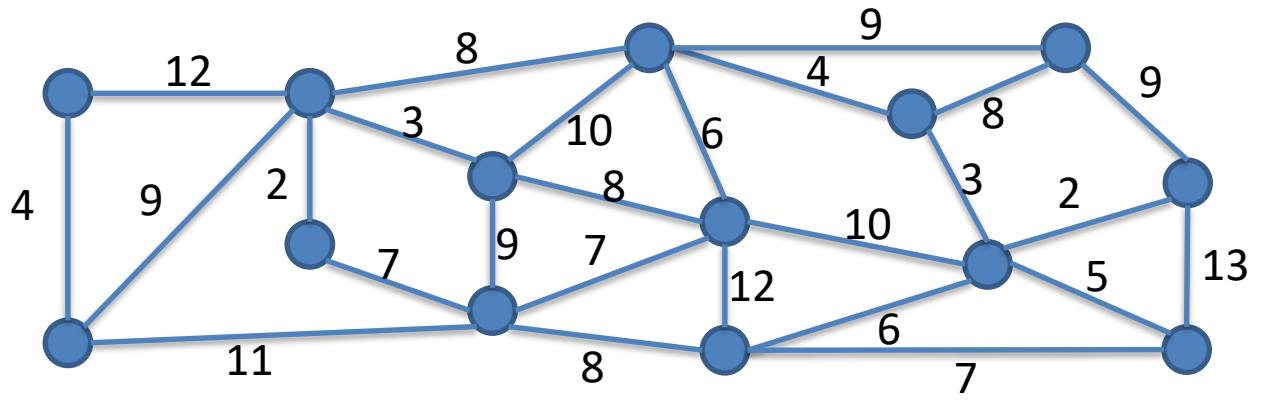
- A minimum spanning tree is a spanning tree with minimum weight.
  - What is the weight of the minimum spanning tree?
  - Is there more than one minimum spanning three for this graph?



Minimum Spanning Trees

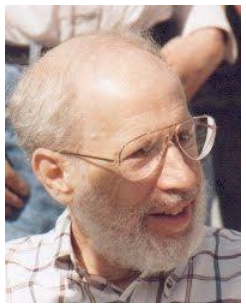
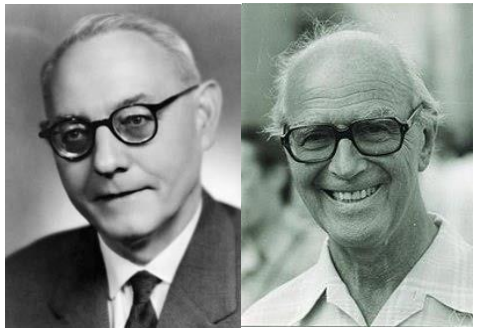
- When we have a weighted graph we can measure the weight of a spanning tree by adding the weights of all edges in the tree.

```
Minimum Spanning Tree (MST)
Input:  G = (V, E, w) a weighted connected undirected graph
Output: m a minimum weight and T = (V, E') a spanning tree of G with weight m
```



Minimum Spanning Trees

- Minimum spanning tree
  - Prim's algorithm
    - Jarnik (1930)
    - Prim (1957)
    - Dijkstra (1959) Turing Award (1972)
  - Kruskal's algorithm
  - Reverse-delete algorithm
    - Kruskal (1956)
  - Boruvka's algorithm
    - Boruvka (1926)
    - Choquet (1938)
    - Sollin (1965)

Cut Property

- In order to prove correctness of minimum spanning tree algorithms a lemma comes in handy.

Definition: Let $G = (V, E)$ be a graph. Let $V_0 \subseteq V$ and $V_1 = V \setminus V_0$. We call this a cut of the graph G. Any edge $(u, v) \in E$ such that $u \in V_0$ and $v \in V_1$ is an edge that crosses the cut.

Lemma: Let $G = (V, E, w)$ be a weighted undirected connected graph. Let V_0 and V_1 be a cut of G. Let $e \in E$ be an edge that crosses the cut with minimum weight. Then there exists a minimum spanning tree of G that includes e.

Proof: Assume we have an arbitrary weighted undirected connected graph $G = (V, E, w)$. Let O be a minimum spanning tree of G.

If O contains e we are done. So assume O does not contain e.

If O does not contain e then adding e to O would create a cycle. This cycle would include another edge e' that crosses the cut. Removing e' would delete the cycle so the resulting graph is a tree. Call this tree O'.

Since e is the minimum weight edge crossing the cut we know $w(e) \leq w(e')$. This means the cost of $O' \leq O$. Since O is a minimum spanning tree we know that the cost of $O \leq O'$. So the cost of $O = O'$. So O' is a minimum spanning tree of G. □

- As a result of this property any algorithm that selects minimum weight edges crossing some cut is optimal.

Prim's Algorithm

Minimum Spanning Tree (MST)
Input: $G = (V, E, w)$ a weighted connected undirected graph
Output: m a minimum weight and $T = (V, E')$ a spanning tree of G with weight m

```
Prim(G = (V, E, w))
        Let T be an empty tree.
        Add v_1 to T.
        While there are vertices not in T.
                Let e = (v, u) be an edge connecting v ∈ T to u ∉ T with minimum weight.
                Add e and u to T.
        End While
        Return T.
End Prim
```

Prim's Algorithm

Theorem: Prim's Algorithm will always find a minimum spanning tree.

Proof: Let $G = (V, E, w)$ be an arbitrary weighted undirected connected graph.

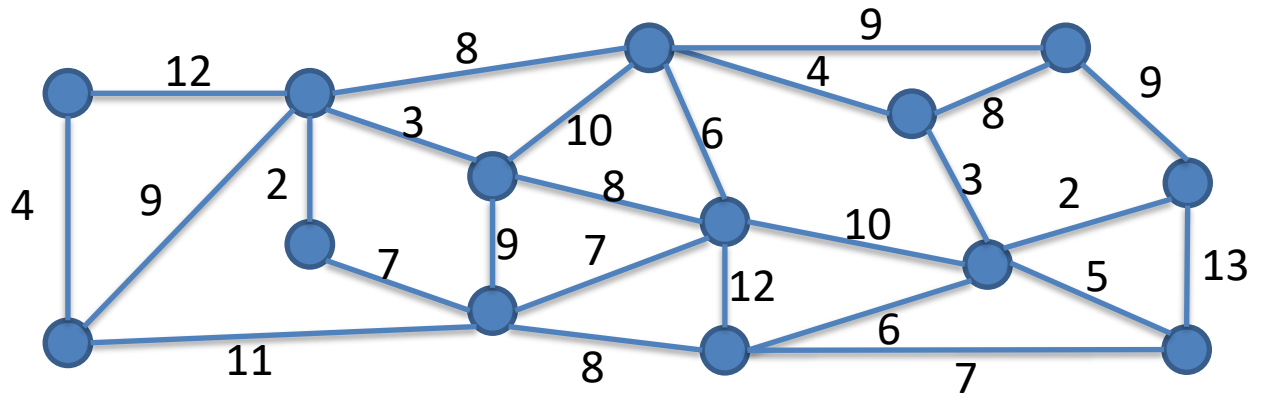
Let T be the tree created by Prim's algorithm.

In each step of Prim's algorithm we connect one vertex that is not in T to T until every vertex has been added to T. This step will never create a cycle, so we know that Prim's algorithm returns a spanning tree.

To see that T is a minimum spanning tree we note that Prim's algorithm is a straightforward application of the cut property. By the lemma, the minimum weight edge connecting any vertex to T is part of some minimum spanning tree of G. Thus T must only include edges in a minimum spanning tree and so T is a minimum spanning tree of G. □

Prim's Algorithm

```
Prim(G = (V, E, w))
        Let T be an empty tree.
        Add v_1 to T.
        While there are vertices not in T.
                Let e = (v, u) be an edge connecting v ∈ T to u ∉ T with minimum weight.
                Add e and u to T.
        End While
        Return T.
End Prim
```



Prim's Algorithm

```
Prim(G = (V, E, w))
        Let T be an empty tree.
        Add v_1 to T.
        While there are vertices not in T.
                Let e = (v, u) be an edge connecting v ∈ T to u ∉ T with minimum weight.
                Add e and u to T.
        End While
        Return T.
End Prim
```

- Prim's algorithm is implemented using a few clever steps to ensure an efficient implementation.
  - First instead of maintaining a set of edges crossing the cut we only maintain a list of vertices connected by edges crossing the cut. Each vertex in this set is labelled with the weight of the minimum weight edge connected to it that crosses the cut.
  - Secondly this set of vertices can be implemented using different data structures: a simple list, a binary heap, or a Fibonacci heap.
- This means that the algorithm consists of two nested loops:

```
        While there are vertices not in T.
                Find the vertex v with the minimum estimate and add v to T along with the minimum weight edge
                        connecting v to T.
                For each vertex u connected to v:
                        If u ∉ T update the estimate of u.
                End For
        End While
```

Prim's Algorithm

```
While there are vertices not in T.
        Find the vertex v with the minimum estimate and add v to T along with the minimum weight edge
                connecting v to T.
        For each vertex u connected to v:
                If u ∉ T update the estimate of u.
        End For
End While
```

- When analyzing the two steps in the loop body it is easier to consider these two steps being carried out in two separate loops:

```
While there are vertices not in T.
        Find the vertex v with the minimum estimate and add v to T along with the minimum weight edge
        connecting v to T.
End While

While there are vertices not in T.
        For each vertex u connected to v:
                If u ∉ T update the estimate of u.
        End For
End While
```

Prim's Algorithm

```
While there are vertices not in T.
        Find the vertex v with the minimum estimate.
End While
```

- The cost of this loop can be expressed by this summation:

$$\sum_{i=1}^{|V|} M(V)$$

- Where $M(V)$ expresses the cost of extracting the vertex with the minimum estimate.
- For a simple list in the worst case $M(V) \in \Theta(|V|)$ but in the best case $M(V) \in \Theta(1)$. As we will see next the total cost of this case is $\Theta(|E|)$.
- For a binary heap or a Fibonacci heap $M(V) \in \Theta(\log|V|)$. So this cost is $\Theta(|V|\log|V|)$.

Prim's Algorithm

```
While there are vertices not in T.
        For each vertex u connected to v:
                If u ∉ T update the estimate of u.
        End For
End While
```

- The cost of this loop can be expressed by this summation:

$$\sum_{v \in V} \sum_{u \in \text{adj}(v)} U(u)$$

- Where $U(u)$ expresses the cost of updating or adding the estimate of u.
- For a simple list and Fibonacci heaps $U(u) \in \Theta(1)$.

$$\sum_{v \in V} \sum_{u \in \text{adj}(v)} d = 2d|E| \in \Theta(|E|)$$

- For a binary heap $U(u) \in \Theta(\log|V|)$.

$$\sum_{v \in V} \sum_{u \in \text{adj}(v)} d \log|V| = 2d|E| \log|V| \in \Theta(|E| \log|V|)$$

Prim's Algorithm

- The runtime of Prim's algorithm depends on the implementation details. This depends most on the data structure used to store the vertices and their minimum cost edges.

- If a simple list is used the runtime of the algorithm is:

$$\Theta(|E|)$$

- If a binary heap is used the runtime of the algorithm is:

$$\Theta\big((|E| + |V|)\log|V|\big) = \Theta(|E|\log|V|)$$

- If a Fibonacci heap is used the runtime of the algorithm is:

$$\Theta(|E| + |V|\log|V|)$$

TCSS 343 – Design and Analysis of Algorithms

Kruskal's Algorithm

Kruskal's Algorithm

Minimum Spanning Tree (MST)
Input: $G = (V, E, w)$ a weighted connected undirected graph
Output: m a minimum weight and $T = (V, E')$ a spanning tree of G with weight m

Kruskal($G = (V, E, w)$)
    Let T be an empty tree.
    For each edge e in ascending order of edge weight.
        Add e (and its endpoints) to T if it will not create a cycle in T.
    End For
    Return T.
End Kruskal

Kruskal's Algorithm

Theorem: Kruskal's Algorithm will always find a minimum spanning tree.

Proof: Let $G = (V, E, w)$ be an arbitrary weighted undirected connected graph.

Let T be the tree created by Kruskal's algorithm.

In each step of Kruskal's algorithm we add an edge (and its endpoints) to T precisely if it does not create a cycle. So we know that Kruskal's algorithm returns a spanning tree.

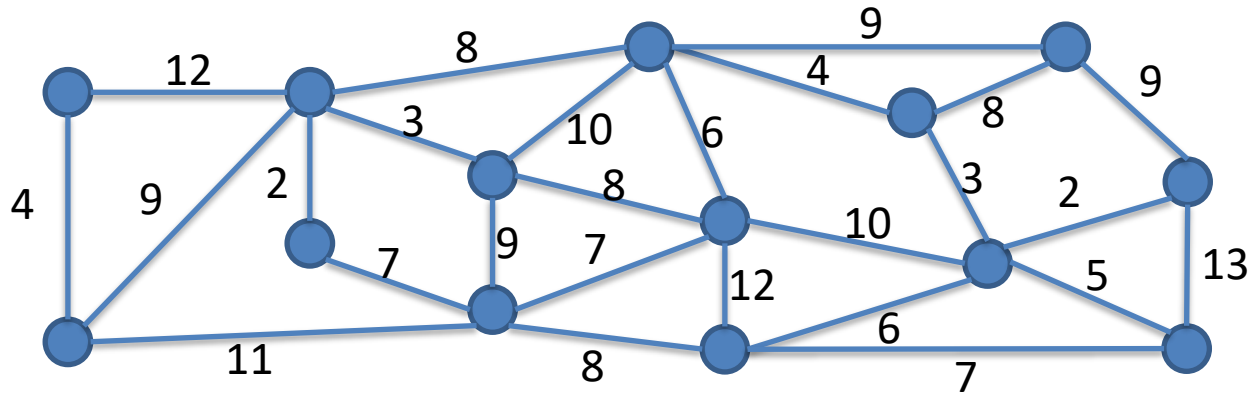
To see that T is a minimum spanning tree we need to see how the cut property applies to Kruskal's algorithm. Consider the edge added at some step of Kruskal's algorithm. Assume it connects vertices u and v.

Since this edge does not create a cycle, we know that u and v are in different subtrees of T at this stage of the algorithm. The subtree that contains v we will call T_v. Consider a cut of G consisting of T_v and $V \setminus T_v$.

Then the edge we add at this step crosses this cut. Since it is the minimum weight edge not already not already added (that will not create a cycle) it must be the minimum weight edge crossing this cut. By the cut property this edge is part of some minimum spanning tree. Thus T must only include edges in a minimum spanning tree and so T is a minimum spanning tree of G. □

Kruskal's Algorithm

```
Kruskal(G = (V, E, w))
        Let T be an empty tree.
        For each edge e in ascending order of edge weight.
                Add e (and its endpoints) to T if it will not create a cycle in T.
        End For
        Return T.
End Kruskal
```



Kruskal's Algorithm

- The runtime of Kruskal's algorithm depends on two steps in the algorithm.
- We must sort the edges:

$$\Theta(|E|\log|E|) = \Theta(|E|\log|V|)$$

- We must iterate through the edges in order of weight:

$$\Theta(|E|) = \Theta(|E|)$$

- So the total runtime of Kruskal's algorithm is:

$$\Theta(|E|\log|V|)$$

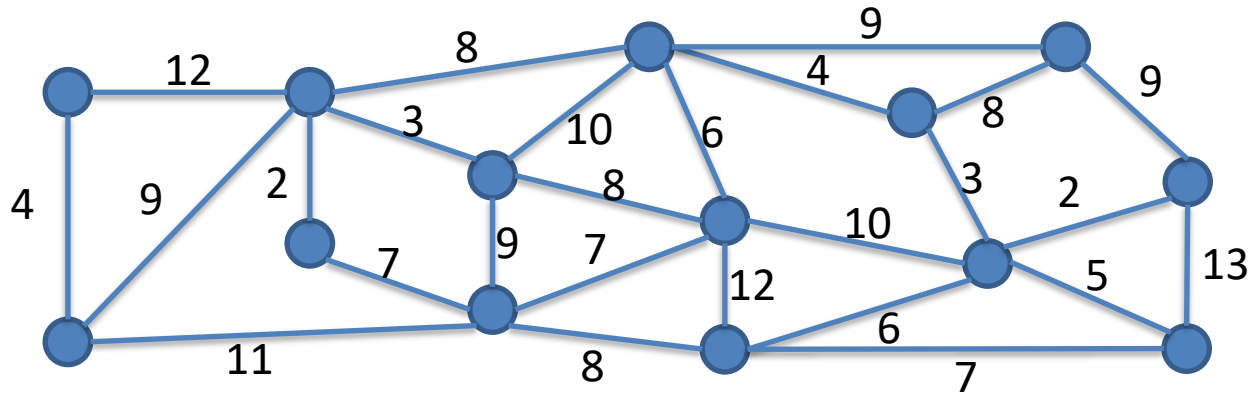
Kruskal's Other Algorithm

Minimum Spanning Tree (MST)
Input: $G = (V, E, w)$ a weighted connected undirected graph
Output: m a minimum weight and $T = (V, E')$ a spanning tree of G with weight m

Kruskal($G = (V, E, w)$)
    Let T be G.
    For each edge e in descending order of edge weight.
        Remove e from T if e is in a cycle in T.
    End For
    Return T.
End Kruskal

Kruskal's Other Algorithm

```
Kruskal(G = (V, E, w))
        Let T be G.
        For each edge e in descending order of edge weight.
                Remove e from T if e is in a cycle in T.
        End For
        Return T.
End Kruskal
```



TCSS 343 – Design and Analysis of Algorithms

Boruvka's Algorithm

Boruvka's Algorithm

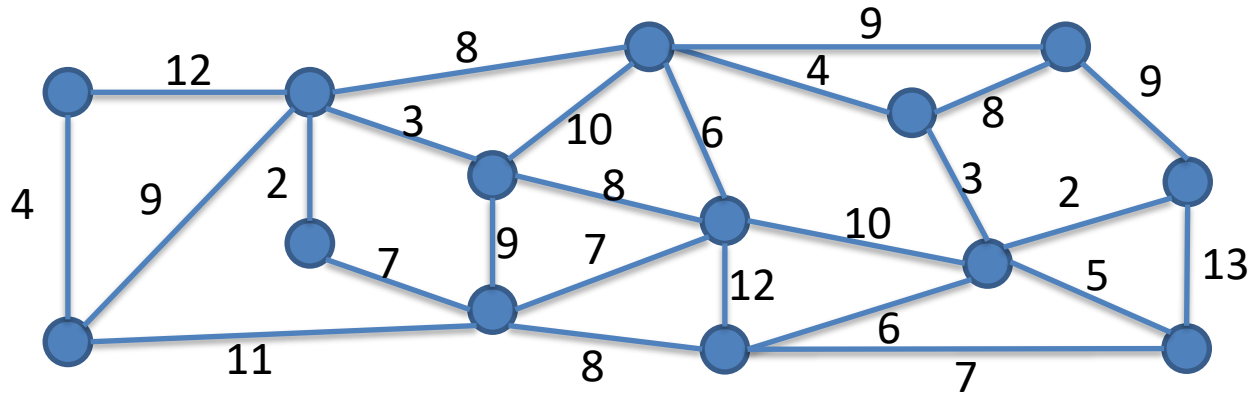
```
Minimum Spanning Tree (MST)
Input: G = (V, E, w) a weighted connected undirected graph
Output: m a minimum weight and T = (V, E') a spanning tree of G with weight m


Boruvka(G = (V, E, w))
        For each vertex v_i create a tree T_i consisting only of that vertex and add it to a queue.
        While the queue has more than one tree:
                Remove the first tree from the queue.  Call it T_i.
                Let e be the edge with minimum weight connecting T_i to any other tree T_j.
                Remove T_j from the queue.
                Merge T_i and T_j using the edge e.  Add the new tree to the queue.
        End While
        Let T be the tree in the queue.
        Return T.
End Boruvka
```

- Like Prim's and Kruskal's algorithm Boruvka's algorithm takes $\Theta(|E|\log|V|)$ operations.

Boruvka's Algorithm

```
Boruvka(G = (V, E, w))
        For each vertex v_i create a tree T_i consisting only of that vertex and add it to a queue.
        While the queue has more than one tree:
                Remove the first tree from the queue.  Call it T_i.
                Let e be the edge with minimum weight connecting T_i to any other tree T_j.
                Remove T_j from the queue.
                Merge T_i and T_j using the edge e.  Add the new tree to the queue.
        End While
        Let T be the tree in the queue.
        Return T.
End Boruvka
```



Boruvka's Algorithm

Theorem: Boruvka's Algorithm will always find a minimum spanning tree.

Proof: Let $G = (V, E, w)$ be an arbitrary weighted undirected connected graph.

Let T be the tree created by Boruvka's algorithm.

In each step of Boruvka's algorithm we connect one subtree T_i to another T_j. This step will never create a cycle, so we know that Boruvka's algorithm returns a spanning tree.

To see that T is a minimum spanning tree we note that Boruvka's algorithm is a straightforward application of the cut property. By the lemma, the minimum weight edge connecting any subtree T_i to another is part of some minimum spanning tree of G. Thus T must only include edges in a minimum spanning tree and so T is a minimum spanning tree of G. □

TCSS 343 – Design and Analysis of Algorithms

Shortest Path in a Graph

Shortest Path In a Graph

- Consider a graph with vertices representing cities and connections representing roads. Given two cities you want to compute the shortest path between the cities.
- Can we formalize this problem as a graph problem?

Shortest Path (SP)
Input: $G = (V, E, w)$ a weighted connected undirected graph and $s \in V$ a source vertex and $d \in V$ a destination vertex and where $w(e) > 0$ for all $e \in E$
Output: m a minimum weight and $P = (p_1, \ldots, p_k)$ a path of edges from s to d where $m = \sum_{i=1}^{k} w(p_i)$

- We will study one solution to this problem developed by Dijkstra in 1956. This algorithm is an example of both dynamic programming and greedy algorithms.
  - Shortest path problem
  - Dijkstra's algorithm (no negative weights)
  - Edsger W. Dijkstra Turing Award 1972
  - Bellman-Ford algorithm (negative weights)
  - A* search algorithm
  - Floyd-Warshall algorithm (all pairs – dynamic programming)

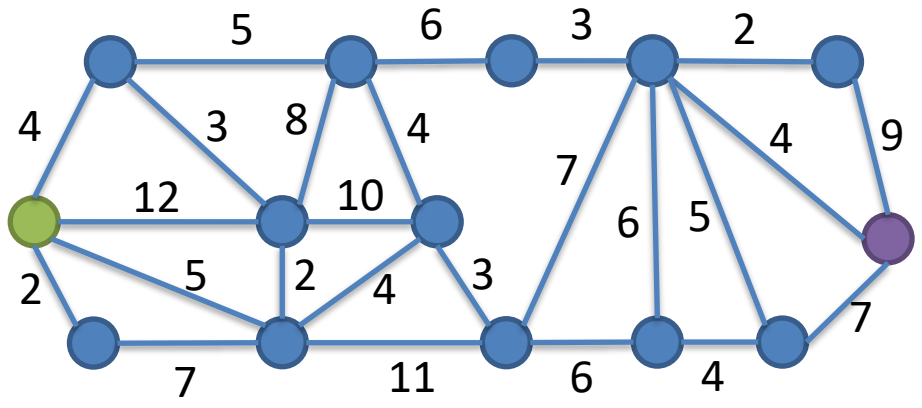


Shortest Path In a Graph

- Can we find a self-reduction from the problem to its sub-problems?
  - Let's fix one end of the path (the source vertex s).
  - Let $SP(d)$ express the length of the shortest path from s to d.

$$SP(d) = \begin{cases} 0 & \text{if } d = s \\ \min_{v \in \text{adj}(d)} \{SP(v) + w(v,d)\} & \text{if } d \neq s \end{cases}$$

- where $\text{adj}(d)$ is the set of vertices adjacent to d and $w(v,d)$ is the weight of the edge from v to d.



Shortest Path In a Graph

- Let $SP(d)$ express the length of the shortest path from s to d.

$$SP(d) = \begin{cases} 0 & \text{if } d = s \\ \min\limits_{v \in \text{adj}(d)} \{SP(v) + w(v, d)\} & \text{if } d \neq s \end{cases}$$

- where $\text{adj}(d)$ is the set of vertices adjacent to d and $w(v, d)$ is the weight of the edge from v to d.

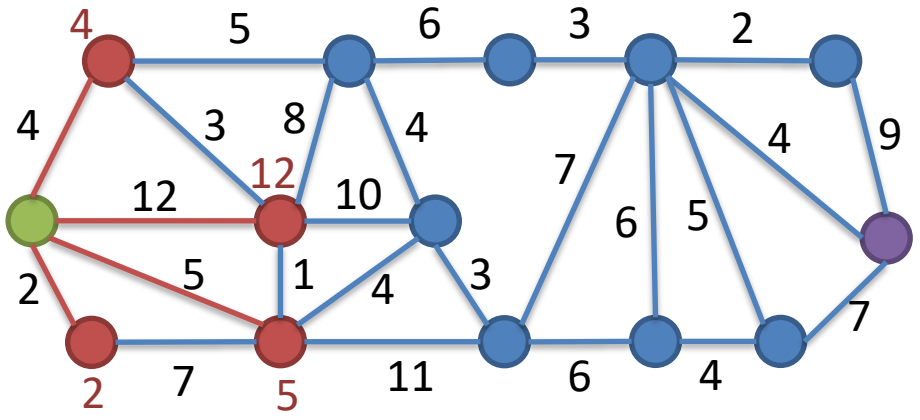
- Is there something wrong with this self-reduction?
  - This self-reduction has infinite overlapping sub-problems.
  - If u is adjacent to v then $SP(u)$ depends on $SP(v)$ and $SP(v)$ is dependant on $SP(u)$.
  - Implementing the top-down divide and conquer algorithm would create an infinite recursion.
  - However, the bottom-up dynamic programming solution might be able to avoid this.

- What data structure would be appropriate?
  - We have one sub-problem per vertex so we could use a 1-d array.
  - It is natural to store the solutions to sub-problems right in the graph.

- What order should we solve the sub-problems?
  - Obviously we should begin with s. However what vertex should we consider next?
  - Time for a greedy choice.

Greedy Choice

- We can estimate the length of the shortest path from s to an adjacent vertex v by using the edge weight $w(s, v)$.
- This graph has estimated path length labelled for the vertices adjacent to s.
  - Which of these estimates are correct and which are false?
  - Which one can we be sure is correct if we don't inspect the rest of the graph?
- The vertex with minimum estimate is correct.
  - Any path that leaves via another edge has greater total weight.
  - This argument assumes there are no negative weights.



Dijkstra's Algorithm

```
Dijkstra(G = (V,E), s, d)
        Let C be an empty set to store vertices with correct shortest paths.
        Let T be an empty set to store vertices with estimated shortest paths.
        Assign s.length = 0.  Add s to C.
        Relax(s).
        Repeat the following steps until d ∈ C.
                Let t ∈ T have the least estimate in T.
                Add t to C.  Remove t from T.
                Relax(t).
        End Repeat
        Return d.length.
End Dijkstra

Relax(u)
        For each vertex v ∉ C adjacent to u calculate an estimated length.
                Let estimate = u.length + w(u,v).
                If v ∉ T assign v.length = estimate. Add v to T.
                If v ∈ T and estimate < v.length assign v.length = estimate.
        End For
End Relax
```

Dijkstra's Algorithm

Theorem: Dijkstra's Algorithm will find a minimum length path from s to d.

Proof: Consider an arbitrary weighted connected undirected graph $G = (V, E, w)$ with no negative weights and $s \in V$ and $d \in V$.

Let S be a list of vertices and the path lengths assigned by Dijkstra's algorithm in the order the vertices are relaxed by Dijkstra's algorithm. Let O be a list of vertices and their shortest path lengths in the same order as S.

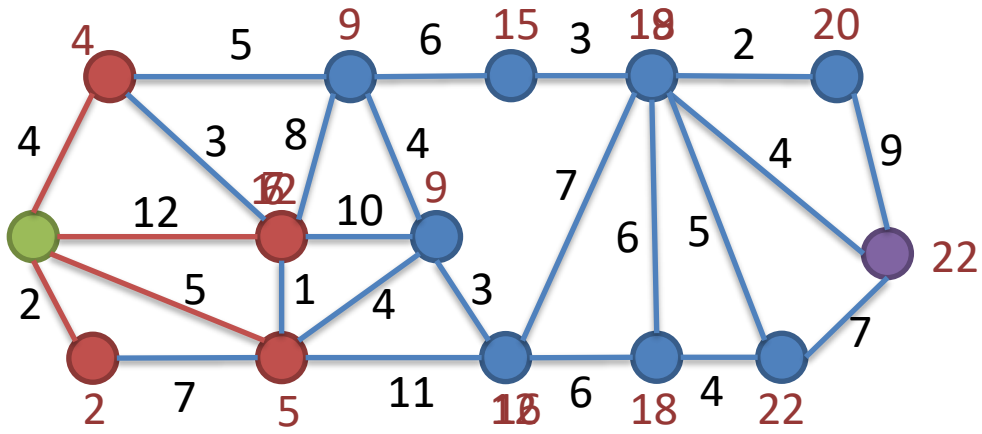
Let v be the first vertex such that the estimate $s_v \neq o_v$. Since O is optimal $o_v < s_v$. Let C be the vertices prior to v upon which S and O agree (this set might only include s).

Consider any path to v other than the one found by S. Any such path must start in C, leave C via another vertex u and then return to v with 0 or more vertices in between.

Since u is connected to a vertex in C it must be a vertex for which we have an estimate. Since v is the vertex with minimum estimate u must have a greater (or equal) estimate to v. Since the cost to reach u is already greater than the cost to reach v we can only offset this with negative weight edges. No negative weight edges occur in the graph so this path must not exist.

Thus the optimal solution must agree with our estimate for v. □

Example: Shortest Path



Dijkstra's Algorithm

- Notice that Dijkstra's algorithm and Prim's algorithm have the exact same structure (Dijkstra independently discovered Prim's algorithm while designing this algorithm).
  - As a result we can analyze the algorithm as though it consists of two separate loops.

- The runtime of Dijkstra's and Prim's algorithm depends on the implementation details. This depends most on the data structure used to store the vertices and their minimum cost edges.

- If a simple list is used the runtime of the algorithm is:

$$\Theta(|E|)$$

- If a binary heap is used the runtime of the algorithm is:

$$\Theta\big((|E| + |V|)\log|V|\big) = \Theta(|E|\log|V|)$$

- If a Fibonacci heap is used the runtime of the algorithm is:

$$\Theta(|E| + |V|\log|V|)$$